

This is my se lab 4. I want you to analyse the files, This is my github link(https://github.com/SETAPESU26/18_blackjack).help me solve the bugs in the game and also identify the bugs in the codes of every file in order to understand the working of the game and also help me fix it.

Worked for 42s

Reviewed SE Lab 4 Blackjack work and recalled assignment tasks

Fixed project logic

Built and tested a fix

I'll help you debug your Software Engineering Lab 4 — Scenario 18: Blackjack vs Dealer step by step. We'll work with your actual code, understand why each bug occurs, fix it, and test that the game behaves correctly rather than replacing your project with an unrelated implementation.

Your lab handout identifies four main tasks:

1. Correct Ace scoring — an Ace must count as 1 or 11 depending on the hand.
2. Complete round resolution — handle blackjack, busts, dealer draws and ties correctly.
3. Implement multi-round bankroll and wagers — maintain chips between rounds and validate bets.
4. Improve feedback and robustness — handle invalid inputs and prevent a round from being settled more than once.

The handout also requires testing boundary cases and submitting short before-and-after gameplay videos, plus the complete LLM chat history. We'll keep those requirements in mind.

1. Analysis of your actual GitHub repository

I inspected your repository: [SETAPESU26/18_blackjack](https://github.com/SETAPESU26/18_blackjack). It contains main.py, game.py, cards.py, requirements.txt, and README.md. The README confirms the four lab tasks listed above.



SETAPESU26/18_blackjack · GitHub

+4

Here is the architecture of your existing game:

main.py

Starts the game

game.py

Player decisions, dealer turns, outcomes and chips

cards.py

Deck creation, shuffling, drawing cards and scoring hands

requirements.txt — declares dependencies

The design is already modular, which is good for your Software Engineering lab. The main problems are in the game rules, round-ending logic and bankroll handling.

2. Bugs identified in each file

File 1 — cards.py

Bug 1: Incorrect Ace scoring — critical

Your current `hand_value()` assigns every Ace a value of 11 and never reduces it to 1 when the hand would exceed 21.



GitHub

Example of the existing bug:

Hand	Current code	Correct value
A + 9	20	20
A + 9 + 5	25 (bust)	15
A + A + 9	31 (bust)	21

Fix: Count Aces as 11 initially, then subtract 10 for each Ace as needed until the hand is at or below 21, or no Aces remain.

Bug 2: Empty-deck handling

`Deck.draw()` returns `None` when the deck is empty. If the caller adds that value to a hand, `hand_value()` cannot unpack it as a (rank, suit) pair.

Fix: Check whether a draw returned `None` before adding the card to a hand. The revised game also reports an empty-deck condition.

Other parts of this file are working as intended: the deck generates 52 distinct rank-and-suit combinations and shuffles them.

File 2 — game.py

This is where most of the gameplay bugs are located.



GitHub

Bug	Why it is a problem	Required fix
Player bust	The round ends before the normal outcome logic, so the original code does not deduct chips for a bust.	Settle the wager as a loss.
Natural blackjack	An opening two-card 21 is not handled as a special outcome.	Detect blackjack before requesting player actions.
Incorrect bankroll design	Chips change by a fixed 10 rather than by the actual wager.	Validate a wager and settle it consistently.
Invalid commands	Inputs other than h, s and q receive no useful feedback.	Display an error and allow another attempt.
Dealer's hidden card	The original display hides both dealer cards instead of showing one face-up card.	Show the first card and conceal the second until reveal.
Round settlement	The flow has several early returns, making it difficult to ensure every outcome updates chips correctly.	Centralize and clearly control round outcomes.
Missing draw feedback	Players don't receive a clear message identifying each newly drawn card.	Print card-draw feedback.
Natural blackjack vs. natural blackjack	The original code does not explicitly resolve this case.	Declare a push and return the wager.

A particularly important bug: In your original code, a player can bust and the method immediately returns. Because the chip adjustment occurs later, that loss is never charged. This can allow a player to keep playing after busting without losing chips.

File 3 — main.py

Your current entry point is simple:

```
from game import Blackjackif __name__ == "__main__": Blackjack().run()
```

There is no major logic bug in this file. It imports the game class and starts it correctly.



GitHub

The main risk is a filename mismatch if you run the originally uploaded copies named `main(3).py` and `game(3).py`. Python expects to find a module called `game` when executing from `game import Blackjack`.

Fix: Keep the filenames `main.py`, `game.py` and `cards.py` together in the same project folder.

File 4 — requirements.txt

Your file specifies that the project uses only Python's standard library and recommends Python 3.9 or newer. No third-party dependencies are required.



GitHub

Finding: No functional bug identified here. You can run the game without `pip install` commands.

File 5 — README.md

Your README describes the lab tasks and testing requirements. One thing to check before submission is whether your actual implementation supports all the features it documents.

The README is not itself a gameplay bug, but documentation must accurately describe the working code. Your submission also requires before-and-after videos and the complete LLM chat-history link.



SETAPESU26/18_blackjack · GitHub

3. Download the corrected project

I created a corrected version of your project while keeping the same core structure and standard-library-only approach.

Blackjack — corrected project

Python source files, README and automated tests

Download the corrected Blackjack project (.zip)

Files included

- main.py
- game.py
- cards.py
- requirements.txt
- README.md
- test_cards.py

What I changed

- cards.py: fixed Ace scoring and retained safe empty-deck behavior.
- game.py: added wager validation, blackjack detection, correct bust handling, dealer card visibility, input feedback and wager refunds for cancelled rounds.
- main.py: retained the simple entry point.
- test_cards.py: added automated tests for hand scoring, multiple Aces, face cards and deck behavior.
- README.md: documented the fixes, how to run the game and the lab testing checklist.

The revised version uses a simple 1:1 payout for wins, including natural blackjack. If your instructor requires the standard 3:2 blackjack payout, that rule should be adjusted.

4. How to run and test the corrected version

1. Download the ZIP and extract it to a folder.
2. Open that extracted folder in VS Code.
3. Open Terminal → New Terminal and run:

```
bash python main.py
```

4. To run the automated tests, execute:

```
bash python -m unittest -v
```

Initial test results

The seven automated tests for card scoring and deck behavior passed in my test run.

Important: these tests do not yet cover every interactive gameplay scenario. The wager flow, dealer decisions and all round outcomes still need additional testing.